

Exercise 1: Basic Unit Test for a Service Method

Service :

```
@Service
public class CalculatorService {

    public int add(int a, int b) {
        return a + b;
    }
}
```

Test :

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CalculatorServiceTest {

    private final CalculatorService calculatorService =
        new CalculatorService();

    @Test
    void testAdd() {

        int result =
            calculatorService.add(10,20);

        assertEquals(30,result);

    }
}
```

Exercise 2: Mocking Repository in Service Test

Service :

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;
```

```
public User getUserById(Long id){  
  
    return userRepository.findById(id)  
        .orElse(null);  
  
}  
  
}
```

Test :

```
import org.junit.jupiter.api.Test;  
import org.junit.jupiter.api.extension.ExtendWith;  
  
import org.mockito.InjectMocks;  
import org.mockito.Mock;  
  
import org.mockito.junit.jupiter.MockitoExtension;  
  
import java.util.Optional;  
  
import static org.junit.jupiter.api.Assertions.*;  
import static org.mockito.Mockito.*;  
  
@ExtendWith(MockitoExtension.class)  
class UserServiceTest {  
  
    @Mock  
    UserRepository repository;  
  
    @InjectMocks  
    UserService service;  
  
    @Test  
    void testGetUser(){  
  
        User user=new User();  
  
        user.setId(1L);  
        user.setName("John");  
  
        when(repository.findById(1L))  
            .thenReturn(Optional.of(user));  
  
        User result=  
            service.getUserById(1L);  
  
        assertEquals("John",  
            result.getName());  
    }  
}
```

```
}  
  
}
```

Exercise 3: Testing Controller using MockMvc

```
import org.junit.jupiter.api.Test;  
  
import org.springframework.beans.factory.annotation.Autowired;  
  
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;  
  
import org.springframework.boot.test.mock.mockito.MockBean;  
  
import org.springframework.test.web.servlet.MockMvc;  
  
import static org.mockito.Mockito.*;  
  
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;  
  
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;  
  
@WebMvcTest(UserController.class)  
class UserControllerTest {  
  
    @Autowired  
    MockMvc mockMvc;  
  
    @MockBean  
    UserService service;  
  
    @Test  
    void testGetUser()  
        throws Exception {  
  
        User user=new User();  
  
        user.setId(1L);  
        user.setName("John");  
  
        when(service.getUserById(1L))  
            .thenReturn(user);  
  
        mockMvc.perform(get("/users/1"))  
  
            .andExpect(status().isOk())
```

```

        .andExpect(jsonPath("$.name")
            .value("John"));
    }
}

```

Exercise 4: Integration Test

```

@SpringBootTest
@AutoConfigureMockMvc
class IntegrationTest {

    @Autowired
    MockMvc mockMvc;

    @Test
    void integrationTest()
        throws Exception {

        mockMvc.perform(get("/users/1"))

            .andExpect(status().isOk());

    }
}

```

Exercise 5: Test POST Endpoint

```

Controller :

@PostMapping
public ResponseEntity<User>
createUser(

    @RequestBody User user){

```

```
return ResponseEntity.ok(
userService.saveUser(user)
);
}
```

Test :

```
@Test
void testCreateUser()
    throws Exception{

    String json="""
        {
            "name":"John"
        }
        """;

    User user=new User();

    user.setId(1L);

    user.setName("John");

    when(service.saveUser(any(User.class)))

        .thenReturn(user);

    mockMvc.perform(

        post("/users")

        .contentType("application/json")

        .content(json)

    )

    .andExpect(status().isOk())

    .andExpect(

        jsonPath("$.name")

        .value("John")

    );
}
```

```
}
```

Exercise 6: Service Exception

```
public User getUser(Long id){  
    return repository.findById(id)  
        .orElseThrow(  
            ()->new NoSuchElementException()  
        );  
}
```

Test :

```
@Test  
void exceptionTest(){  
    when(repository.findById(1L))  
        .thenReturn(Optional.empty());  
    assertThrows(  
        NoSuchElementException.class,  
        ()->{  
            service.getUser(1L);  
        }  
    );  
}
```

Exercise 7: Repository Query

Repository :

```
public interface UserRepository  
extends JpaRepository<User,Long>{
```

```
List<User>
```

```
findByName(String name);
```

```
}
```

Test :

```
@DataJpaTest
```

```
class RepositoryTest {
```

```
    @Autowired
```

```
    UserRepository repository;
```

```
    @Test
```

```
    void testFindByName(){
```

```
        User user=new User();
```

```
        user.setName("John");
```

```
        repository.save(user);
```

```
        List<User> users=
```

```
            repository.findByName("John");
```

```
        assertEquals(
```

```
            1,
```

```
            users.size()
```

```
        );
```

```
    }
```

```
}
```

Exercise 8: Exception Handler

@ControllerAdvice

```
public class GlobalExceptionHandler{

    @ExceptionHandler(
        NoSuchElementException.class)

    public ResponseEntity<String>

    handle(){

        return ResponseEntity

            .status(404)

            .body("User not found");

    }

}
```

Test

```
@Test
void exceptionController()
    throws Exception{

    when(service.getUserById(1L))

        .thenThrow(

            new NoSuchElementException()

        );

    mockMvc.perform(get("/users/1"))

        .andExpect(status()

            .isNotFound())

        .andExpect(content()

            .string("User not found"));

}
```

Exercise 9: Parameterized Test

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest{

    CalculatorService service=

        new CalculatorService();

    @ParameterizedTest
    @CsvSource({
        "2,3,5",
        "5,5,10",
        "10,15,25"
    })
    void testAdd(
        int a,
        int b,
        int expected){
        assertEquals(
            expected,
            service.add(a,b)
        );
    }
}
```

